



Projet de design d'un matériau composite

Auteurs

Mariame DIA

AbdouRahman FALL

Souleymane NIANE

DIC3

Aerospace engineering students

Enseignant:

Commandant Abdoulaye SARR

Date : **07 Janvier 2025**
Ecole Polytechnique de Thiès
Thiès, Sénégal

Contents

0.1	Introduction	2
0.2	Choix des matériaux	2
0.3	Hypothèses et fondements théoriques	3
0.3.1	Hypothèses liées à la théorie des laminés	3
0.3.2	Critère de Tsai-Hill	4
0.3.3	Symétrie imposée	4
0.3.4	Autres hypothèses simplificatrices	4
0.4	Algorithme	4
0.4	Script python	7
0.4.1	Initialisation des plies	7
0.4.2	Calcul de la matrice de rigidité réduite	8
0.4.3	Calcul de la matrice de rigidité réduite transformée	8
0.4.4	Calcul des matrices A, B et D	9
0.4.5	Calcul des contraintes dans les axes globaux et locaux	10
0.4.6	Calcul du critère de Tsi-Hill	12
0.4.7	Ajustement des angles	13
0.4.8	Ajouter des plies	14
0.4.9	Calcul des propriétés du stratifié	16
0.4.10	Affichage des résultats	16
0.4.11	Programme final	19
0.5	Application : Reservoir sous pression	22
0.5.1	Interprétation des résultats obtenus pour les différents matériaux	23
0.5.2	Résumé des performances	24
0.5.3	Recommandations pour le choix du matériau	24
0.6	Conclusion	24

0.1 Introduction

Dans un monde où les exigences en termes de performance et de durabilité des matériaux ne cessent de croître, les composites occupent une place centrale grâce à leur capacité à combiner légèreté, résistance mécanique et adaptabilité aux besoins spécifiques. Ces matériaux innovants permettent de répondre efficacement aux défis des secteurs tels que l'aéronautique, l'automobile, ou encore les structures civiles.

Dans le cadre de notre projet, nous nous intéressons à la conception et à l'analyse d'un matériau composite optimisé pour une structure soumise à des charges. L'objectif principal est de déterminer les **propriétés mécaniques de ce composite, d'évaluer les contraintes qu'il peut supporter et de définir un design optimal**. Cela inclut notamment l'étude de l'orientation des plis, de l'empilement des couches, et de l'épaisseur adéquate, afin d'assurer à la fois une performance mécanique optimale et une utilisation efficiente des ressources. Cette démarche s'appuie sur une combinaison d'approches théoriques et numériques par la réalisation d'un programme en Python afin de maîtriser les étapes clés du design des matériaux composites.

0.2 Choix des matériaux

Dans cette section, nous présentons les choix de matériaux pour la conception de notre matériau composite. Afin d'optimiser les performances mécaniques et de répondre aux contraintes de la structure étudiée, nous avons sélectionné trois combinaisons de fibres et matrices, chacune offrant des propriétés spécifiques adaptées à différents scénarios d'application :

1. **E-Glass Epoxy** : Ce composite est largement utilisé pour sa bonne résistance mécanique, sa légèreté et son coût abordable. Il est idéal pour des applications nécessitant un équilibre entre performance et coût.

Table 1: Properties of E-glass Fiber/Epoxy Composite

Property	Notation	E-glass Fiber/Epoxy
Young's Modulus	E_x (GPa)	36.5
	$E_y = E_z$ (GPa)	15
Poisson's Ratio	$\nu_{xy} = \nu_{xz}$	0.24
	ν_{yz}	0.22
Shear Modulus	G_{xy} (GPa)	6.35
	G_{yz} (GPa)	1.6
Tensile Strength	X_t (MPa)	1050
	$Y_t = Z_t$ (MPa)	43
Compression Strength	X_c (MPa)	-938
	$Y_c = Z_c$ (MPa)	-106
Shear Strength	$S_{xy} = S_{yz} = S_{xz}$ (MPa)	88

2. **Carbon Epoxy** : Reconnue pour sa rigidité et sa résistance exceptionnelle, cette combinaison est choisie pour les structures nécessitant des performances élevées, en particulier dans les environnements soumis à des contraintes intenses.

Table 2: Properties of Carbon Fiber/Epoxy Composite

Property	Notation	Carbon Fiber/Epoxy
Young's Modulus	E_x (GPa)	127.7
	$E_y = E_z$ (GPa)	7.4
Poisson's Ratio	$\nu_{xy} = \nu_{xz}$	0.33
	ν_{yz}	0.188
Shear Modulus	G_{xy} (GPa)	6.9
	G_{yz} (GPa)	4.3
Tensile Strength	X_t (MPa)	1717
	$Y_t = Z_t$ (MPa)	30
Compression Strength	X_c (MPa)	-1200
	$Y_c = Z_c$ (MPa)	-216
Shear Strength	$S_{xy} = S_{yz} = S_{xz}$ (MPa)	33

3. **Aluminium Silicon Carbide** : Ce composite à matrice métallique (MMC) offre une excellente résistance à l'usure, une conductivité thermique élevée et une bonne résistance mécanique, le rendant adapté aux applications du domaine aéronautiques et spatiales.

Table 3: Properties of DWA 6092 SiCp Composite

Property	Value
Young Modulus ($E_x = E_y$, isotropic)	101 GPa
Shear Modulus	39 GPa
Poisson's Ratio	0.29
Ultimate Tensile Strength	476 MPa
Ultimate Compression Strength	500 MPa

Ces choix nous permettront d'explorer et de comparer les comportements mécaniques de ces composites dans différents contextes d'application.

Dans cette étude, nous allons concevoir le matériau avec une épaisseur de pli uniforme de **0,3 mm**, quel que soit le type de matériau utilisé.

0.3 Hypothèses et fondements théoriques

Dans cette section, nous exposons les hypothèses fondamentales adoptées pour la conception et l'analyse de notre matériau composite. Ces hypothèses constituent le socle méthodologique qui oriente les calculs, et la validation des performances mécaniques.

0.3.1 Hypothèses liées à la théorie des laminés

La théorie classique des laminés repose sur les hypothèses suivantes :

Déformation linéaire

Les déformations à travers l'épaisseur du stratifié sont supposées linéaires.

Absence de glissement inter-plis :

Les plis sont parfaitement liés, sans séparation ni glissement.

Matériaux orthotrope

Chaque pli est considéré comme un matériau orthotrope.

Comportement en petites déformations :

Les déformations et rotations sont supposées petites, permettant d'utiliser des relations linéaires.

Les formules classiques de la théorie laminés, telles qu'elles sont présentées dans le support de cours, ont été utilisées

0.3.2 Critère de Tsai-Hill

Le critère de Tsai-Hill a été utilisé pour évaluer la sécurité mécanique des plis sous contrainte. Ce critère, qui repose sur une relation quadratique des contraintes, est défini comme suit :

$$\left(\frac{\sigma_1}{X_t}\right)^2 - \frac{\sigma_1\sigma_2}{X_t^2} + \left(\frac{\sigma_2}{Y_t}\right)^2 + \left(\frac{\tau_{12}}{S}\right)^2 \leq 1$$

Où :

- X_t : résistance à la traction dans la direction 1.
- Y_t : résistance à la traction dans la direction 2.
- S : résistance au cisaillement.

Justification du choix :

Le critère de Tsai-Hill est largement utilisé dans les études sur les matériaux composites car il prend en compte les interactions entre contraintes dans différentes directions tout en restant simple à appliquer.

0.3.3 Symétrie imposée

Pour garantir un comportement mécanique optimal, la symétrie des plis a été imposée. Cela inclut :

1. Disposition symétrique :

Les plis sont disposés de manière symétrique par rapport au plan moyen du stratifié.

2. Avantages de la symétrie :

- Découplage entre les réponses en flexion et en membrane.
- Réduction des déformations dues aux charges thermiques ou hygroscopiques.

0.3.4 Autres hypothèses simplificatrices

Afin de simplifier les calculs et accélérer le processus de conception, les hypothèses suivantes ont été adoptées:

1. Les effets de fatigue, d'usure ou de vieillissement du matériau ne sont pas pris en compte.
2. Les contraintes thermiques sont ignorées.
3. Les tolérances manufacturières et les défauts éventuels dans les plis (ex. porosité, alignement imparfait) ne sont pas considérés.

0.4 Algorithme

L'algorithme pour la conception d'un stratifié composite repose sur une approche itérative visant à optimiser les angles et le nombre de plis, afin de garantir la résistance mécanique du matériau en utilisant le critère de Tsai-Hill. Ce critère permet d'évaluer la sécurité mécanique des plis sous les charges appliquées. L'objectif

principal est de concevoir un matériau capable de supporter ces charges tout en minimisant son poids, dans une optique de design structurel optimal.

La démarche commence par le choix d'un matériau composite, dont les propriétés mécaniques telles que les modules d'élasticité, les coefficients de Poisson et les limites de résistance sont définies. Ensuite, les forces et moments appliqués sont spécifiés afin de simuler les conditions réelles d'utilisation. À partir des caractéristiques intrinsèques du matériau, la matrice de rigidité réduite est calculée, cette dernière étant indépendante de l'orientation des plis. Le stratifié est initialisé avec un nombre minimal de plis imposé par des considérations technologiques, auxquels sont attribués des angles d'orientation initiaux.

Une caractéristique importante de cet algorithme est la gestion symétrique des plis. Cette symétrie permet de découpler les réponses en flexion et en membrane tout en évitant le gauchissement dû aux charges thermiques. Chaque pli situé dans la partie supérieure du stratifié possède un équivalent symétrique dans la partie inférieure. Les angles des plis sont ajustés symétriquement afin de rechercher des configurations qui minimisent les contraintes locales tout en respectant le critère de Tsai-Hill. Si aucune orientation ne satisfait ce critère, deux plis supplémentaires sont ajoutés de manière symétrique pour renforcer la structure.

Le processus est répété de manière itérative jusqu'à ce que tous les plis respectent le critère de Tsai-Hill. Une fois la configuration optimale atteinte, les contraintes locales et globales sont recalculées pour chaque pli, et les propriétés mécaniques globales du stratifié, telles que les modules d'élasticité et les coefficients de Poisson, sont évaluées dans le repère global. Les résultats finaux incluent les contraintes par pli, la séquence d'empilement, les orientations optimales et les propriétés mécaniques globales du stratifié. Cette méthodologie garantit un stratifié robuste, performant et adapté aux conditions de service prévues.

Ci-dessous, un diagramme illustrant clairement l'algorithme, offrant une meilleure compréhension du processus et facilitant ainsi la visualisation des différentes étapes et interactions impliquées.

Algorithme de design

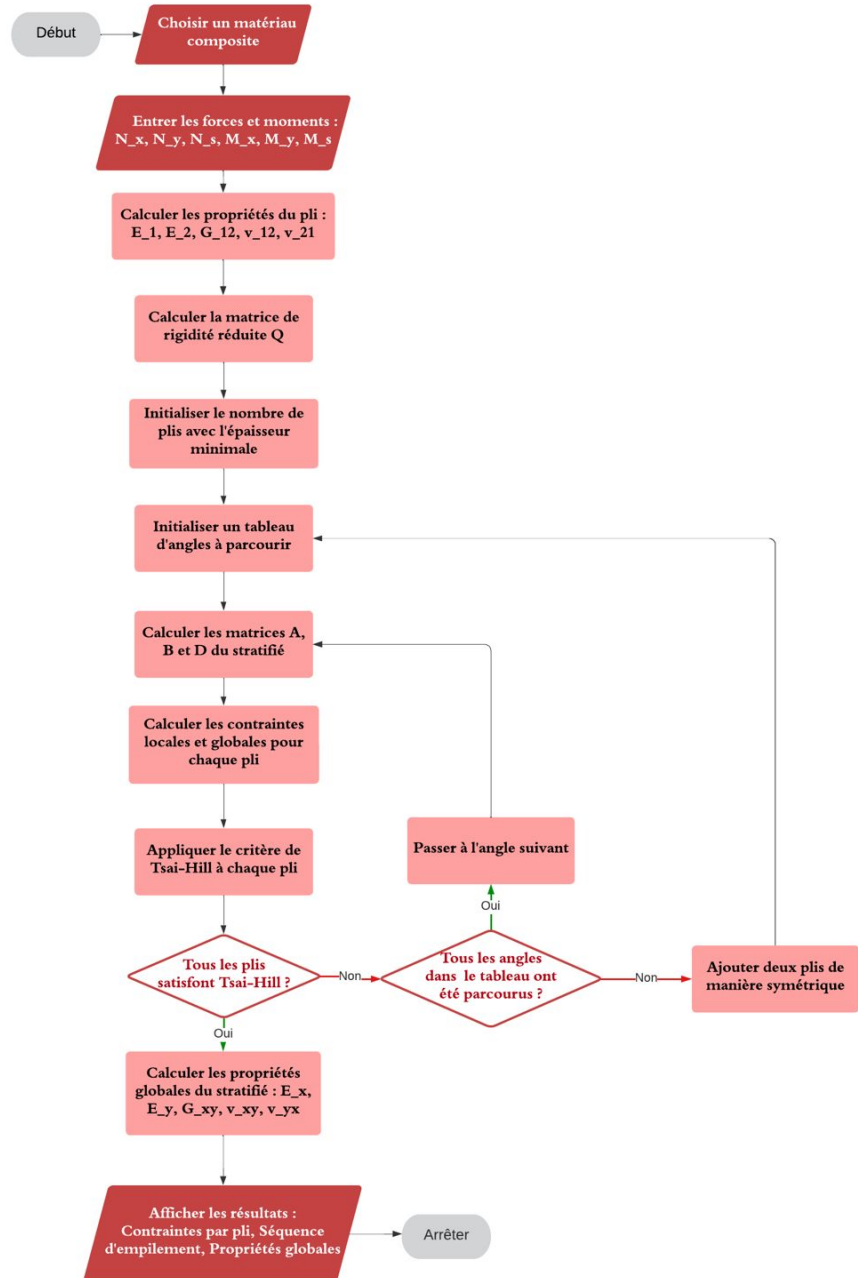


Figure 1: Algorithme de design

0.4 Script python

Cette section présente le script Python utilisé pour concevoir le matériau composite. [4] Chaque bloc de code est accompagné d'une brève description de son fonctionnement et de commentaires intégrés pour en faciliter la compréhension.

0.4.1 Initialisation des plis

Cette fonction initialise un stratifié composite symétrique en respectant une épaisseur minimale imposée par des contraintes technologiques. Elle suit trois étapes clés :

1. Calcul du nombre minimal de plis : Le nombre de plis nécessaires pour chaque moitié du stratifié est déterminé à partir de l'épaisseur minimale totale spécifiée et de l'épaisseur d'un pli.
2. Construction symétrique : Les plis sont disposés symétriquement par rapport à l'axe médian $z = 0$. Ils sont d'abord ajoutés au-dessus de cet axe, puis en miroir en dessous, pour garantir un équilibre mécanique.
3. Définition des propriétés des plis : Chaque pli est décrit par:
 - ses positions (z_{top} , z_{bottom})
 - son épaisseur
 - son angle d'orientation
 - ses contraintes et déformations (globales et locales) via des dictionnaires (`global_strain`, `global_stress`, `local_stress`).

```
[361]: import numpy as np

def initialise_number_ply(ply_thickness, angles=30, min_thickness=1e-3):
    """
    Initialise un stratifié symétrique avec une épaisseur minimale,
    en positionnant l'axe médian au niveau \ ( z = 0 \ ).

    Paramètres :
    -----
    ply_thickness : float
        Épaisseur de chaque pli (en mm).
    angles : float
        Angle d'orientation des plis (symétrie imposée).
    min_thickness : float
        Épaisseur minimale totale du stratifié (par défaut 1.0 mm).

    Retour :
    -----
    ply_results : list of dict
        Liste des plis avec `z_top`, `z_bottom`, et `angles` configurés symétriquement.
    """
    # Nombre minimal de plis dans chaque moitié
    num_plies_half = int(np.ceil(min_thickness / (2 * ply_thickness)))
    total_thickness = 2 * num_plies_half * ply_thickness

    # Construction des plis symétriques
    ply_results = []
    z_top = total_thickness / 2

    for _ in range(2 * num_plies_half):
```

```

        z_bottom = z_top - ply_thickness
        ply_results.append({
            "z_top": z_top,
            "z_bottom": z_bottom,
            "thickness": ply_thickness,
            "angle": angles,
            "global_strain": 0,
            "global_stress": 0,
            "local_stress": 0,
        })
        z_top = z_bottom

    return ply_results

```

0.4.2 Calcul de la matrice de rigidité réduite

La fonction `calculate_Q_matrix` calcule la matrice de rigidité réduite Q pour un pli composite. 1. Les entrées sont les propriétés du matériaux a savoir: E_1 , E_2 , G_{12} , ν_{12} , ν_{21} 2. Les éléments de la matrice Q sont calculés avec les formules standards : 3. En sortie la fonction retourne un dictionnaire contenant les termes de la matrice: Q_{11} , Q_{22} , Q_{12} , Q_{66} . [5]

```

[404]: def calculate_Q_matrix(E_1, E_2, G_12, v_12):
        """
        Calcule la matrice de rigidité réduite Q pour un ply.
        """
        v_21 = (E_2 / E_1) * v_12
        Q_11 = E_1 / (1 - v_12 * v_21)
        Q_22 = E_2 / (1 - v_12 * v_21)
        Q_12 = v_12 * E_2 / (1 - v_12 * v_21)
        Q_66 = G_12

        return {
            'Q11': Q_11,
            'Q22': Q_22,
            'Q12': Q_12,
            'Q66': Q_66,
        }

```

0.4.3 Calcul de la matrice de rigidité réduite transformée

La fonction `calculate_transformed_Q` calcule les coefficients de rigidité transformés (Q'_{ij}) pour un pli orienté selon un angle donné (θ).

```

[405]: def calculate_laminate_matrices(ply_data, Q_matrix):
        """
        Calcule les matrices A, B et D pour un stratifié composite.

        Paramètres :
        -----
        ply_data : list of dict
            Liste contenant les informations pour chaque pli :
            - 'angle': float, angle d'orientation du pli (en degrés)
            - 'z_top': float, position de la surface supérieure du pli
            - 'z_bottom': float, position de la surface inférieure du pli

```

```

    Q_matrix : dict
        Dictionnaire des coefficients de la matrice de rigidité réduite Q (non_
↳ transformée).

    Retour :
    -----
    A, B, D : np.ndarray
        Matrices A (rigidité membrane), B (couplage membrane-flexion),
        et D (rigidité flexion).
    """
    # Initialisation des matrices A, B et D (3x3)
    A = np.zeros((3, 3))
    B = np.zeros((3, 3))
    D = np.zeros((3, 3))

    # Calcul des matrices A, B et D en parcourant chaque pli
    for ply in ply_data:
        angle = ply['angle']
        z_top, z_bottom = ply['z_top'], ply['z_bottom']
        delta_z = z_top - z_bottom

        # Matrice Q transformée pour l'angle du pli
        Q_transformed = calculate_transformed_Q(Q_matrix, angle)
        Q_trans = np.array([
            [Q_transformed['Q11'], Q_transformed['Q12'], Q_transformed['Q16']],
            [Q_transformed['Q12'], Q_transformed['Q22'], Q_transformed['Q26']],
            [Q_transformed['Q16'], Q_transformed['Q26'], Q_transformed['Q66']]
        ])

        # Contributions aux matrices A, B et D
        A += Q_trans * delta_z
        B += Q_trans * (z_top**2 - z_bottom**2) / 2
        D += Q_trans * (z_top**3 - z_bottom**3) / 3

    return A, B, D

```

0.4.4 Calcul des matrices A, B et D

Cette fonction calcule les matrices A, B ET D qui sont appelées matrices de rigidité en extension, en couplage et en flexion, respectivement pour un stratifié composite avec les formules usuels

```

[406]: def calculate_laminate_matrices(ply_data, Q_matrix):
    """
        Calcule les matrices A_ij, B_ij, D_ij pour un stratifié composite.

        Paramètres :
        -----
        ply_data : list of dict
            Liste contenant les informations pour chaque pli, sous forme de dictionnaires_
↳ avec :
            - 'Q_transformed': dict avec Q11', Q22', Q12', Q66', Q16', Q26'
            - 'z_top': float, position de la surface supérieure
            - 'z_bottom': float, position de la surface inférieure
    """

```

```

        - 'thickness': float, épaisseur du pli"""

# Initialisation des matrices A, B et D
A = np.zeros((3, 3))
B = np.zeros((3, 3))
D = np.zeros((3, 3))

# Parcours des plis pour accumuler les contributions
for ply in ply_data:
    angle = ply['angle']

    # Matrice Q transformée
    Q_transformed = calculate_transformed_Q(Q_matrix, angle)
    z_top, z_bottom = ply['z_top'], ply['z_bottom']
    delta_z = z_top - z_bottom

    # Construction de la matrice Q transformée
    Q_trans = np.array([
        [Q_transformed['Q11'], Q_transformed['Q12'], Q_transformed['Q16']],
        [Q_transformed['Q12'], Q_transformed['Q22'], Q_transformed['Q26']],
        [Q_transformed['Q16'], Q_transformed['Q26'], Q_transformed['Q66']]
    ])

    # Contribution aux matrices A, B et D
    A += Q_trans * delta_z
    B += Q_trans * (z_top**2 - z_bottom**2) / 2
    D += Q_trans * (z_top**3 - z_bottom**3) / 3

return A, B, D

```

0.4.5 Calcul des contraintes dans les axes globaux et locaux

Cette fonction calcule les contraintes et déformations dans un pli spécifique d'un stratifié composite sous l'effet de forces et moments avec l'aide des formules usuels. Les résultats incluent les déformations globales, les contraintes globales et locales.

```

[407]: def calculate_stresses_laminate(A, B, D, N, M, ply, Q_matrix):
    """
    Calcule les contraintes et déformations dans un pli d'un stratifié composite.

    Paramètres :
    -----
    A, B, D : ndarray
        Matrices de rigidité du stratifié.
    N : ndarray
        Forces normales globales [N_x, N_y, N_xy].
    M : ndarray
        Moments globaux [M_x, M_y, M_xy].
    ply : dict
        Informations sur le pli :
        - 'z_top': position supérieure,
        - 'z_bottom': position inférieure,
        - 'angle': angle d'orientation du pli (en degrés).
    """

```

```

Q_matrix : dict
    Matrice de rigidité réduite du matériau (ex : {'Q11': ..., 'Q12': ..., etc.}).

Retour :
-----
ply : dict
    Mise à jour des informations du pli avec les déformations et contraintes_
    ↪ calculées :
        - 'global_strain': déformations globales,
        - 'global_stress': contraintes globales,
        - 'local_stress': contraintes locales.
    """
    # Construction de la matrice globale de rigidité ABD
    ABD = np.block([
        [A, B],
        [B, D]
    ])

    # Construction du vecteur des charges
    loads = np.concatenate([N, M]) # [N_x, N_y, N_xy, M_x, M_y, M_xy]

    # Résolution pour obtenir les déformations moyennes ( $\epsilon$ ) et courbures ( $\kappa$ )
    strains_and_curvatures = np.dot(np.linalg.inv(ABD), loads)
    mid_plane_strains = strains_and_curvatures[:3] #  $\epsilon$ 
    mid_plane_curvatures = strains_and_curvatures[3:] #  $\kappa$ 

    # Calcul des déformations globales dans le pli
    z_mid = (ply["z_top"] + ply["z_bottom"]) / 2
    global_strain = mid_plane_strains + z_mid * mid_plane_curvatures

    # Matrice Q transformée pour l'angle du pli
    angle = ply["angle"]
    Q_transformed = calculate_transformed_Q(Q_matrix, angle) # Retourne un_
    ↪ dictionnaire

    # Construction de la matrice \(( Q )\) transformée à partir du dictionnaire
    Q_matrice = np.array([
        [Q_transformed["Q11"], Q_transformed["Q12"], Q_transformed["Q16"]],
        [Q_transformed["Q12"], Q_transformed["Q22"], Q_transformed["Q26"]],
        [Q_transformed["Q16"], Q_transformed["Q26"], Q_transformed["Q66"]]
    ])

    # Contraintes globales dans le pli
    global_stress = np.dot(Q_matrice, global_strain)

    # Transformation des contraintes globales vers locales
    theta = np.radians(angle)
    c, s = np.cos(theta), np.sin(theta)

    # Matrice de transformation T
    T = np.array([
        [c**2, s**2, -2*c*s],
        [s**2, c**2, 2*c*s],

```

```

        [c*s, -c*s, c**2 - s**2]
    ])

    # Contraintes locales
    local_stress = np.dot(np.linalg.inv(T), global_stress)

    # Enregistrement des résultats
    ply["global_strain"] = global_strain
    ply["global_stress"] = global_stress
    ply["local_stress"] = local_stress

    return ply

```

0.4.6 Calcul du critère de Tsi-Hill

La fonction `tsai_hill_criterion` implémente le critère de Tsai-Hill, un modèle utilisé pour évaluer la sécurité des plis dans un matériau composite en tenant compte de ses résistances à la traction, à la compression, et au cisaillement.

```

[408]: def tsai_hill_criterion(local_stress, Xt, Xc, Yt, Yc, S):
    """
    Vérifie le critère de Tsai-Hill pour un matériau composite.

    Contraintes locales dans le pli ( $\sigma_x$ ,  $\sigma_y$ ,  $\tau_{xy}$ ).
    ↪
    Xt, Xc : float
        Résistances en traction et en compression dans la direction longitudinale.
    Yt, Yc : float
        Résistances en traction et en compression dans la direction transverse.
    S : float
        Résistance au cisaillement dans le plan.

    Retour :
    -----
    critereon : float
        Valeur du critère de Tsai-Hill (doit être <= 1 pour que le pli soit sûr).
    """
    sigma_x, sigma_y, tau_xy = local_stress

    # Choisir la résistance en fonction du signe de sigma_x
    X = Xt if sigma_x >= 0 else Xc
    # Choisir la résistance en fonction du signe de sigma_y
    Y = Yt if sigma_y >= 0 else Yc

    # Calcul du critère
    critereon = (
        (sigma_x / X) ** 2
        - (sigma_x * sigma_y) / (X ** 2) # Utilise Xt et Xc directement ici
        + (sigma_y / Y) ** 2
        + (tau_xy / S) ** 2
    )

    return critereon

```

0.4.7 Ajustement des angles

La fonction `adjust_ply_angles` ajuste les angles des plis d'un stratifié composite symétrique pour satisfaire le critère de Tsai-Hill, garantissant ainsi la sécurité du matériau sous des charges données. Les angles sont optimisés pour minimiser les risques de rupture. Un point clé ici est que un pli *i* et son symétrie *j* sont ajustés avec le même angle pour conserver la symétrie du stratifié.

```
[409]: def adjust_ply_angles(ply_data, Q_matrix, N, M, limit_strength):
        """
        Ajuste les angles des plis dans un stratifié symétrique pour satisfaire le critère_
        ↪ de Tsai-Hill.

        Paramètres :
        -----
        ply_data : list of dict
            Liste contenant les données des plis.
        Q_matrix : ndarray
            Matrice de rigidité réduite.
        N : ndarray
            Forces normales (N_x, N_y, N_xy).
        M : ndarray
            Moments (M_x, M_y, M_xy).
        limit_strength : dict
            Limites de résistance mécanique du matériau.

        Retour :
        -----
        ply_data : list of dict
            Liste mise à jour des plis avec les angles ajustés.
        """
        angles = [0, -15, 15, -30, 30, -45, 45, -60, 60, -75, 75, 90] # Angles disponibles
        num_plies = len(ply_data)
        mid_index = num_plies // 2 # Indice de l'axe médian (si symétrie)

        for i in range(mid_index):
            # Initialiser les critères pour le pli i (au-dessus) et le pli j (en dessous)
            j = num_plies - i - 1 # Pli symétrique
            ply_i = ply_data[i]
            ply_j = ply_data[j]
            criteria_i = []
            criteria_j = []

            for angle in angles:
                # Ajuste l'angle du pli i et symétriquement du pli j
                ply_i['angle'] = angle
                ply_j['angle'] = angle

                # Recalculer les matrices A, B, D
                A, B, D = calculate_laminate_matrices(ply_data, Q_matrix)

                # Calculer les contraintes locales pour les plis i et j
                stresses_i = calculate_stresses_laminate(A, B, D, N, M, ply_i, Q_matrix)
                stresses_j = calculate_stresses_laminate(A, B, D, N, M, ply_j, Q_matrix)
```

```

local_stress_i = stresses_i['local_stress']
local_stress_j = stresses_j['local_stress']

# Calculer les valeurs du critère de Tsai-Hill pour i et j
criterion_i = tsai_hill_criterion(
    local_stress_i,
    limit_strength["Xt"], limit_strength["Xc"],
    limit_strength["Yt"], limit_strength["Yc"], limit_strength["S"]
)
criterion_j = tsai_hill_criterion(
    local_stress_j,
    limit_strength["Xt"], limit_strength["Xc"],
    limit_strength["Yt"], limit_strength["Yc"], limit_strength["S"]
)

# Enregistrer la pire valeur entre les deux plis pour ce choix d'angle
criteria_i.append(max(criterion_i, criterion_j))

# Décision pour les angles des plis i et j
criteria_i = np.array(criteria_i)
if np.all(criteria_i > 1): # Aucun angle ne satisfait le critère
    closest_index = np.argmin(np.abs(criteria_i - 1))
    ply_i["angle"] = angles[closest_index]
    ply_j["angle"] = angles[closest_index] # Symétrie
else: # Plusieurs angles satisfont le critère
    valid_indices = np.where(criteria_i <= 1)[0]
    min_index = valid_indices[np.argmin(criteria_i[valid_indices])]
    ply_i["angle"] = angles[min_index]
    ply_j["angle"] = angles[min_index] # Symétrie

# Recalculer les matrices A, B, D
A, B, D = calculate_laminate_matrices(ply_data, Q_matrix)

# Calculer les contraintes locales pour les plis i et j
stresses_i = calculate_stresses_laminate(A, B, D, N, M, ply_i, Q_matrix)
stresses_j = calculate_stresses_laminate(A, B, D, N, M, ply_j, Q_matrix)

local_stress_i = stresses_i['local_stress']
local_stress_j = stresses_j['local_stress']

ply_i["global_strain"] = stresses_i["global_strain"]
ply_i["global_stress"] = stresses_i["global_stress"]
ply_i["local_stress"] = stresses_i["local_stress"]

ply_j["global_strain"] = stresses_j["global_strain"]
ply_j["global_stress"] = stresses_j["global_stress"]
ply_j["local_stress"] = stresses_j["local_stress"]

return ply_data

```

0.4.8 Ajouter des plis

La fonction `add_ply` permet d'ajouter une paire de plis symétriques à une liste existante de plis dans un stratifié composite. Voici une description des principaux points. L'objectif majeur est la maintien de la

symétrie du stratifié en ajoutant un pli supérieur et un pli inférieur, de manière à ne pas modifier la position de l'axe médian

```
[410]: def add_ply(ply_data, thickness):  
    """  
    Ajoute une paire de plis symétriques pour respecter la symétrie du stratifié  
    sans nécessiter de réajustement du mid-plane.  
  
    Paramètres :  
    -----  
    ply_data : list of dict  
        Liste des plis existants dans le stratifié.  
    thickness : float  
        Épaisseur de chaque pli à ajouter.  
  
    Retour :  
    -----  
    ply_data : list of dict  
        Liste mise à jour des plis avec la paire ajoutée.  
    """  
  
    # Ajouter une paire symétrique par rapport à l'axe médian  
    z_bottom_upper = ply_data[0]["z_top"]  
    z_top_upper = z_bottom_upper + thickness  
  
    # Ajouter le pli supérieur  
    ply_data.insert(0, {  
        "z_top": z_top_upper,  
        "z_bottom": z_bottom_upper,  
        "thickness": thickness,  
        "angle": 0,  
        "global_strain": np.zeros(3),  
        "global_stress": np.zeros(3),  
        "local_stress": np.zeros(3)  
    })  
  
    z_top_lower = ply_data[-1]["z_bottom"]  
    z_bottom_lower = z_top_lower - thickness  
  
    # Ajouter le pli inférieur  
    ply_data.append({  
        "z_top": z_top_lower,  
        "z_bottom": z_bottom_lower,  
        "thickness": thickness,  
        "angle": 0,  
        "global_strain": np.zeros(3),  
        "global_stress": np.zeros(3),  
        "local_stress": np.zeros(3)  
    })  
  
    return ply_data
```

0.4.9 Calcul des propriétés du stratifié

Cette fonction `calculate_laminate_properties` calcule les propriétés mécaniques effectives d'un stratifié composite à partir de sa matrice de rigidité A .

```
[411]: def calculate_laminate_properties(A_matrix, thickness_total):
        """
        Calcule les propriétés effectives du stratifié composite.

        Paramètres :
        - A_matrix : ndarray
            Matrice de rigidité A du stratifié (issue des matrices A, B, D).
        - thickness_total : float
            Épaisseur totale du stratifié.

        Retourne :
        - properties : dict
            Dictionnaire contenant les propriétés  $(E_x)$ ,  $(E_y)$ ,  $(G_{xy})$ ,  $(\nu_{xy})$ , et  $(\nu_{yx})$ .
        """
        # Inversion de la matrice A pour obtenir la souplesse
        compliance_matrix = np.linalg.inv(A_matrix)

        # Calcul des propriétés
        E_x = 1 / (compliance_matrix[0, 0] * thickness_total)
        E_y = 1 / (compliance_matrix[1, 1] * thickness_total)
        G_xy = 1 / (compliance_matrix[2, 2] * thickness_total)
        v_xy = -compliance_matrix[0, 1] / compliance_matrix[0, 0]
        v_yx = v_xy * (E_y / E_x) # Relation entre v_xy et v_yx

        return {
            "E_x": E_x,
            "E_y": E_y,
            "G_xy": G_xy,
            "v_xy": v_xy,
            "v_yx": v_yx
        }
```

0.4.10 Affichage des résultats

Ces fonctions suivantes permettent l'affichage des propriétés du stratifiés incluant une validation Tsai-Hill et la séquence d'empilement. La bibliothèque `pandas` a été utilisé à cet effet

```
[412]: def generate_stacking_sequence(ply_data):
        """
        Génère une séquence d'empilement standardisée avec les indices inférieurs pour les plis consécutifs.

        Arguments :
        - ply_data : Liste de dictionnaires contenant l'angle des plis.

        Retourne :
        - Une chaîne de caractères représentant la séquence d'empilement.
        """
```

```

def to_subscript(number):
    """
    Convertit un entier en indice (subscript) Unicode.
    """
    subscript_map = str.maketrans("0123456789", "")
    return str(number).translate(subscript_map)

angles = [ply["angle"] for ply in ply_data] # Récupération des angles
half_sequence = angles[:len(angles) // 2 + len(angles) % 2] # Partie unique
↳ (avant symétrie)

# Comptage des plis consécutifs au même angle
sequence = []
current_angle = half_sequence[0]
count = 0

for angle in half_sequence:
    if angle == current_angle:
        count += 1
    else:
        if count > 1:
            sequence.append(f"{current_angle}{to_subscript(count)}")
        else:
            sequence.append(f"{current_angle}")
            current_angle = angle
            count = 1
if count > 1:
    sequence.append(f"{current_angle}{to_subscript(count)}")
else:
    sequence.append(f"{current_angle}")

# Assemblage final avec le souscrit `` pour indiquer la symétrie
return f"[{ ' / '.join(sequence) }]"

```

```

[418]: def display_results(ply_data, material_properties, limit_strength):
    """
    Affiche les résultats avec validation Tsai-Hill, séquence d'empilage et propriétés
    ↳ des matériaux.
    """
    # Prepare results data
    results_data = []
    for idx, ply in enumerate(ply_data, start=1):
        tsai_hill = tsai_hill_criterion(
            ply["local_stress"],
            limit_strength["Xt"], limit_strength["Xc"],
            limit_strength["Yt"], limit_strength["Yc"],
            limit_strength["S"]
        )
        results_data.append({
            "Pli": idx,
            "Orientation": f"{ply['angle']}°",
            "z_top (m)": f"{ply['z_top']:.3e}",
            "z_bottom (m)": f"{ply['z_bottom']:.3e}",

```

```

        "σ (Pa)": f"{ply['local_stress'][0]:.2e}",
        "σ (Pa)": f"{ply['local_stress'][1]:.2e}",
        "τ (Pa)": f"{ply['local_stress'][2]:.2e}",
        "Tsai-Hill": f"{tsai_hill:.2f}",
        "Validation": "OK" if tsai_hill <= 1 else f"{Fore.RED}NON{Style.RESET_ALL}"
    })

# Convert results to DataFrame
df_results = pd.DataFrame(results_data)

# Generate stacking sequence
stacking_sequence = generate_stacking_sequence(ply_data)
total_plies = len(ply_data)

# Create a new DataFrame for material properties and stacking info
summary_data = [
    ["E (Pa)", f"{material_properties['E_x']:.2e}"],
    ["E (Pa)", f"{material_properties['E_y']:.2e}"],
    ["G (Pa)", f"{material_properties['G_xy']:.2e}"],
    ["ν", f"{material_properties['ν_xy']:.3f}"],
    ["ν", f"{material_properties['ν_yx']:.3f}"],
    ["Nombre total de plis", total_plies],
    ["Séquence d'empilage", stacking_sequence],
]

df_summary = pd.DataFrame(summary_data, columns=["", ""]) # Create empty headers

# Display Results
display(HTML(
    f"""
        <div style="display: flex; flex-direction: column; align-items: center; width: 100%;">
            <div style="text-align: center; color: brown; font-weight: bold; font-size: 18px; margin-bottom: 20px;">
                {"=" * 50}<br>
                PROPRIÉTÉS DES PLI<br>
                {"=" * 50}
            </div>
            <div style="width: 100%; display: flex; justify-content: center; margin-bottom: 20px;">
                {df_results.style.set_table_styles([
                    {'selector': 'th', 'props': [('text-align', 'center'), ('font-size', '16px'), ('font-weight', 'bold')]}],
                    {'selector': 'td', 'props': [('text-align', 'center'), ('font-size', '14px')]}
                ]).hide(axis='index').to_html()}
            </div>
            <div style="text-align: center; color: brown; font-weight: bold; font-size: 18px; margin-bottom: 20px;">
                {"=" * 50}<br>
                RÉSUMER GLOBAL<br>
                {"=" * 50}
        </div>
    """

```

```

        </div>
        <div style="width: 100%; display: flex; justify-content: center;
↪margin-bottom: 20px;">
            {df_summary.style.set_table_styles([
                {'selector': 'th', 'props': [('display', 'none')]},
                {'selector': 'td', 'props': [('text-align', 'center')],
↪('font-size', '16px'), ('font-weight', 'bold')]}
            ].hide(axis='index').set_table_attributes('style="width: 150%;").
↪to_html())}
        </div>
    </div>
    """
))

```

0.4.11 Programme final

Ce code implément les différentes fonctions sus-mentionnées en simulant le processus d'analyse et d'optimisation d'un stratifié en se basant sur les propriétés des matériaux, les efforts appliqués, et les moments de flexion. Voici une explication des différentes parties :

1. **Définition des matériaux composites :**

Un dictionnaire `composites` contient les propriétés mécaniques et de résistance de trois matériaux composites.

2. **Interface utilisateur :**

L'utilisateur sélectionne le matériau composite et saisit les données nécessaires (forces et moments).

3. **Calculs principaux :**

- **Matrice de rigidité Q :** Calculée à l'aide des propriétés du matériau (via `calculate_Q_matrix`).
- **Initialisation des plis :** Les plis sont initialisés avec une épaisseur donnée (via `initialise_number_ply`).
- **Ajustement des angles des plis :** Une optimisation itérative ajuste les angles des plis pour satisfaire le critère de Tsai-Hill (via `adjust_ply_angles`).
- **Ajout de plis :** Si les contraintes dépassent les limites, des paires de plis symétriques sont ajoutées (via `add_ply`).
- **Propriétés globales du stratifié :** Calcul des propriétés mécaniques globales (E_x , E_y , G_{xy} , etc.) avec `calculate_laminate_properties`.

4. **Affichage des résultats :**

- **Propriétés des plis :** Contraintes locales et validation du critère de Tsai-Hill.
- **Résumé global :** Propriétés mécaniques et séquence d'empilement (via `display_results`).

```

[423]: composites = {
    1: {
        "name": "Composite E-Glass Fiber/Epoxy",
        "E_1": 36.5e9, # Module d'élasticité longitudinale (Pa)
        "E_2": 15e9, # Module d'élasticité transverse (Pa)
        "v_12": 0.24, # Coefficient de Poisson dans la direction longitudinale
        "G_12": 6.35e9, # Module de cisaillement (Pa)
        "thickness": 0.3e-3, # Épaisseur d'un pli (m)
        "Xt": 1050e6, # Résistance en traction longitudinale (Pa)
        "Xc": -938e6, # Résistance en compression longitudinale (Pa)
    }
}

```

```

        "Yt": 43e6,      # Résistance en traction transverse (Pa)
        "Yc": -106e6,   # Résistance en compression transverse (Pa)
        "S": 88e6       # Résistance au cisaillement (Pa)
    },
    2: {
        "name": "Composite Carbone Fiber/Epoxy",
        "E_1": 127.7e9,
        "E_2": 7.4e9,
        "v_12": 0.33,
        "G_12": 6.9e9,
        "thickness": 0.3e-3,
        "Xt": 1717e6,
        "Xc": -1200e6,
        "Yt": 30e6,
        "Yc": -216e6,
        "S": 33e6
    },
    3: {
        "name": "Composite SiC Aluminium/Epoxy",
        "E_1": 101e9,
        "E_2": 101e9,
        "v_12": 0.29,
        "G_12": 39e9,
        "thickness": 0.1e-3,
        "Xt": 476e6,
        "Xc": 500e6,
        "Yt": 476e6,
        "Yc": 500e6,
        "S": 303e6
    }
}

# Interface utilisateur pour choisir le matériaux composite
print("Choisissez un matériaux composite :")
for key, value in composites.items():
    print(f"{key}. {value['name']}")

while True:
    try:
        materiaux_choice = int(input("Votre choix de matériaux composite (1, 2, 3) :"))
        ↵
        if materiaux_choice in [1, 2, 3]:
            materiaux = composites[materiaux_choice]
            break
        else:
            print("Choix invalide. Veuillez entrer 1, 2 ou 3.")
    except ValueError:
        print("Veuillez entrer un nombre valide.")

# Entrées supplémentaires
'''while True:
    try:
        fiber_volume_fraction = float(

```

```

        input("Entrez la fraction volumique de fibre (entre 0 et 1) : ")
    if 0 <= fiber_volume_fraction <= 1:
        break
    else:
        print("Veuillez entrer une valeur entre 0 et 1.")
except ValueError:
    print("Veuillez entrer un nombre valide.")'''

# Forces normales
while True:
    try:
        N = list(
            map(float, input("Entrez les forces (N_x N_y N_s) : ").split()))
        if len(N) == 3:
            break
        else:
            print("Veuillez entrer exactement 3 valeurs.")
    except ValueError:
        print("Veuillez entrer des nombres valides (séparés par des espaces).")

# Moments
while True:
    try:
        M = list(map(float, input("Entrez les moments (M_x M_y M_s) : ").split()))
        if len(M) == 3:
            break
        else:
            print("Veuillez entrer exactement 3 valeurs.")
    except ValueError:
        print("Veuillez entrer des nombres valides (séparés par des espaces).")

E_1, E_2, v_12, G_12 = materiaux["E_1"], materiaux["E_2"], materiaux["v_12"],
↪materiaux["G_12"]
e = materiaux["thickness"]
limit_strength = {"Xt": materiaux["Xt"], "Xc": materiaux["Xc"], "Yt": materiaux["Yt"],
↪"Yc": materiaux["Yc"], "S": materiaux["S"]}

Q_matrix = calculate_Q_matrix(E_1, E_2, G_12, v_12)

ply_data = initialise_number_ply(e)

while True:
    ply_data = adjust_ply_angles(ply_data, Q_matrix, N, M, limit_strength)
    if all(tsai_hill_criterion(ply["local_stress"], limit_strength["Xt"],
↪limit_strength["Xc"], limit_strength["Yt"], limit_strength["Yc"], limit_strength["S"]),
↪<= 1 for ply in ply_data):
        break
    ply_data = add_ply(ply_data, e)

A, *Z = calculate_laminate_matrices(ply_data, Q_matrix)

# Calculate material properties
material_properties = calculate_laminate_properties(A, e)

```

```
display_results(ply_data, material_properties, limit_strength)
```

0.5 Application : Reservoir sous pression

Dans cette partie, nous allons appliqué notre programme à un cas pratique afin d'optimiser le design d'un matériau composite soumis à des chargements spécifiques.

Pour cela, nous avons choisi de travailler avec les données de l'exercice **7.16 du livre Engineering Mechanics of Composite Materials par Isaac M. Daniel et Ori Ishai [2]**. Cet exercice porte sur le design d'un réservoir sous pression (pressure vessel) en matériaux composites.

Les données collectées sont les suivantes :

$$N_x = 460 \text{ kN/m,}$$

$$N_y = 920 \text{ kN/m,}$$

$$N_s = 228 \text{ kN/m}$$

L'utilisateur choisit un materiau puis entre les forces et moments comme indiqué sur le bloc ci-dessous.

Choisissez un matériau composite :

1. Composite E-Glass Fiber/Epoxy
2. Composite Carbone Fiber/Epoxy
3. Composite SiC Aluminium/Epoxy

Votre choix de matériaux composite (1, 2, 3) : 2

Entrez les forces (N_x N_y N_s) : 460000 920000 228000

Entrez les moments (M_x M_y M_s) : 0 0 0

Les résultats suivants sont obtenus pour le **Carbon fiber/Epoxy** :

=====								
PROPRIÉTÉS DES PLI								
=====								
Pli	Orientation	z_top (m)	z_bottom (m)	σ_1 (Pa)	σ_2 (Pa)	τ_{12} (Pa)	Tsai-Hill	Validation
1	0°	2.100e-03	1.800e-03	2.07e+08	2.12e+07	1.69e+07	0.78	OK
2	0°	1.800e-03	1.500e-03	2.07e+08	2.12e+07	1.69e+07	0.78	OK
3	45°	1.500e-03	1.200e-03	5.59e+08	1.58e+06	-1.96e+06	0.11	OK
4	90°	1.200e-03	9.000e-04	3.03e+08	1.74e+07	-1.69e+07	0.63	OK
5	90°	9.000e-04	6.000e-04	3.03e+08	1.74e+07	-1.69e+07	0.63	OK
6	90°	6.000e-04	3.000e-04	3.03e+08	1.74e+07	-1.69e+07	0.63	OK
7	90°	3.000e-04	0.000e+00	3.03e+08	1.74e+07	-1.69e+07	0.63	OK
8	90°	0.000e+00	-3.000e-04	3.03e+08	1.74e+07	-1.69e+07	0.63	OK
9	90°	-3.000e-04	-6.000e-04	3.03e+08	1.74e+07	-1.69e+07	0.63	OK
10	90°	-6.000e-04	-9.000e-04	3.03e+08	1.74e+07	-1.69e+07	0.63	OK
11	90°	-9.000e-04	-1.200e-03	3.03e+08	1.74e+07	-1.69e+07	0.63	OK
12	45°	-1.200e-03	-1.500e-03	5.59e+08	1.58e+06	-1.96e+06	0.11	OK
13	0°	-1.500e-03	-1.800e-03	2.07e+08	2.12e+07	1.69e+07	0.78	OK
14	0°	-1.800e-03	-2.100e-03	2.07e+08	2.12e+07	1.69e+07	0.78	OK

Figure 2: Orientation et séquence des plies

RÉSUMER GLOBAL	
E_x (Pa)	6.44e+11
E_y (Pa)	1.12e+12
G_{xy} (Pa)	2.06e+11
ν_{xy}	0.108
ν_{yx}	0.187
Nombre total de plis	14
Séquence d'empilage	$[0_2 / 45 / 90_4]_S$

Figure 3: Propriétés mécaniques

0.5.1 Interprétation des résultats obtenus pour les différents matériaux

1. Composite E-Glass Fiber/Epoxy

a. Résultats de conception :

- Nombre de plis : 32
- Séquence d'empilement : $[90/75/90/75/90/75_2/90/75/90/75_4/90/75]_S$
- La séquence indique une conception complexe nécessitant un nombre élevé de plis pour atteindre une résistance suffisante.

b. Commentaire :

- Le grand nombre de plis (32) peut engendrer un surcroît de masse et compliquer la fabrication.
- Cependant, ce design offre un bon compromis entre le coût modéré des matériaux et les performances mécaniques globales.

2. Composite Carbone Fiber/Epoxy

a. Résultats de conception :

- Nombre de plis : 14
- Séquence d'empilement : $[0_2/45/90_4]_S$
- La séquence plus simple montre que ce matériau nécessite moins de plis pour atteindre une performance mécanique élevée.

b. Commentaire :

- Ce composite est idéal pour des applications où un poids léger et une résistance élevée sont requis.
- Avec seulement 14 plis, il présente une excellente performance, tout en réduisant considérablement la masse.
- Néanmoins, le coût élevé des matériaux à base de fibre de carbone pourrait constituer une contrainte.

3. Composite SiC Aluminium/Epoxy

a. Résultats de conception :

- Nombre de plis : 20
- Séquence d'empilement : $[0_{10}]_S$

- La structure d'empilement est simple et axée sur une orientation uniforme.

b. Commentaire :

- Ce composite est performant pour des applications nécessitant une résistance accrue au cisaillement et une bonne stabilité thermique.
- Bien qu'il soit plus lourd que les composites à base de fibre de carbone, il offre un bon compromis pour les applications où les contraintes en cisaillement dominent sur les contraintes en traction ou en compression.

0.5.2 Résumé des performances

a. Composite E-Glass Fiber/Epoxy :

- Avantages : Séquence d'empilement complexe mais équilibrée, coût relativement bas.
- Inconvénients : Poids plus élevé et complexité accrue dans la fabrication.

b. Composite Carbone Fiber/Epoxy :

- Avantages : Léger, haute résistance, faible nombre de plis.
- Inconvénients : Coût élevé des matériaux.

c. Composite SiC Aluminium/Epoxy :

- Avantages : Excellente résistance au cisaillement, bonne stabilité thermique, structure simple.
- Inconvénients : Masse relativement élevée par rapport à la fibre de carbone.

0.5.3 Recommandations pour le choix du matériau

1. Poids léger et haute performance :

Le composite **Carbone Fiber/Epoxy** est la solution optimale pour minimiser la masse tout en maximisant les performances mécaniques.

2. Comportement équilibré en termes de coût et performance :

Le composite **E-Glass Fiber/Epoxy** est adapté si le budget est une priorité, tout en maintenant un bon niveau de résistance.

3. Exigences spécifiques de résistance au cisaillement et stabilité thermique :

Le composite **SiC Aluminium/Epoxy** est idéal pour des environnements nécessitant une stabilité thermique ou une résistance accrue aux forces de cisaillement.

0.6 Conclusion

En conclusion, ce projet a permis de poser les bases d'une conception innovante de matériaux composites, parfaitement adaptés aux exigences des structures soumises à des forces complexes. L'utilisation d'un programme en Python pour déterminer les propriétés mécaniques du matériau ainsi que sa configuration a été essentielle pour définir une approche de design optimisée. Ce travail fut l'opportunité de développer des compétences solides en programmation informatique et de mieux assimiler les notions théoriques du cours. Enfin, il pourrait servir de base pour des recherches futures dans le domaine des matériaux composites.

Bibliography

- [1] Ahmed W. Abdel-Ghany and Samy J. Ebeid Iman Taha. “Failure Prediction of Fiber Reinforced Polymer Pipes using FEA”. In: *International Journal of Engineering and Technical Research (IJETR)* (February 2016).
- [2] Isaac M. Daniel and Ori Ishai. *Engineering Mechanics of Composite Materials*. Oxford University Press, 1994.
- [3] Matweb material property data. *DWA 6092/SiC/17.5p-T6 Aluminum SiC Metal*. URL: <https://www.matweb.com/search/DataSheet.aspx?MatGUID=179aaeb42a7346ddba6f9172b93c4109>.
- [4] NPTEL IIT Guwahati. *Mechanics of Fiber Reinforced Polymer Composite Structures*. URL: <https://www.youtube.com/playlist?list=PLwdnzlV3ogoVE2AIC-G4Uew8XsaINwJGo>.
- [5] Commandant Abdoulaye SARR. *Composite Lecture*.